Foundations of Language Modeling

Chapter 2: Building a Character-Level Model

1 The Bigram Language Model

Before building a complex Transformer, we begin with the simplest possible neural language model: the **Bigram Model**. Recall that a general language model computes $P(x_t \mid x_1, \ldots, x_{t-1})$. The bigram model simplifies this by making a strong Markov assumption.

Definition 1.1 (Bigram Markov Assumption)**.** *A bigram language model assumes that the probability of a token x_t depends strictly and exclusively on the immediately preceding token x_{t-1}. Formally:*

$$P(x_t \mid x_1, x_2, \ldots, x_{t-1}) \approx P(x_t \mid x_{t-1}) \tag{1}$$

Intuition: The model has no "memory" of tokens prior to step $t-1$. If the model sees the sequence "The quick brown", it will predict the next word based entirely on the word "brown", completely ignoring "The quick".

2 Token Embeddings and Logits

In PyTorch, the bigram model is implemented using an `nn.Embedding` layer. Mathematically, an embedding layer is simply a lookup table, equivalent to a learnable matrix.

Definition 2.1 (Embedding Matrix and Logits)**.** *Let $V = |\mathcal{V}|$ be the vocabulary size. We define an embedding matrix $\mathbf{W} \in \mathbb{R}^{V \times V}$. Given an input token $x_{t-1} = i$, the model retrieves the i-th row of $\mathbf{W}$. This row vector, denoted $\mathbf{l} \in \mathbb{R}^V$, contains the **logits** (unnormalized log-probabilities) for the next token:*

$$\mathbf{l} = \mathbf{W}_{i,*} \tag{2}$$

Remark 2.2 (Tensor Dimensions)**.** *When processing a batch of data, the input is a tensor $\mathbf{X} \in \{0, \ldots, V-1\}^{B \times T}$, where B is the batch size and T is the block size. Applying the embedding layer yields a logit tensor $\mathbf{L} \in \mathbb{R}^{B \times T \times C}$, where C is the channel dimension. In a pure bigram model, $C = V$.*

3 Model Evaluation: Cross-Entropy Loss

To evaluate how well the model's logits predict the true next tokens, we use the **Cross-Entropy Loss**, which relies on the Softmax function to convert logits into a valid probability distribution.

Definition 3.1 (Softmax Function)**.** *Given a vector of logits $\mathbf{l} = (l_1, \ldots, l_V)$, the Softmax function $\sigma : \mathbb{R}^V \to (0,1)^V$ is defined for the k-th component as:*

$$p_k = \sigma(\mathbf{l})_k = \frac{\exp(l_k)}{\sum_{j=1}^{V} \exp(l_j)} \tag{3}$$

Definition 3.2 (Cross-Entropy / Negative Log-Likelihood)**.** *Let $y \in \{1, \dots, V\}$ be the true target token class, and $\mathbf{p} = \sigma(\mathbf{l})$ be the predicted probability distribution. The cross-entropy loss $\mathcal{L}$ for a single prediction is the negative log-probability of the true target class:*

$$\mathcal{L}(\mathbf{l}, y) = -\log(p_y) = -l_y + \log\left(\sum_{j=1}^{V} \exp(l_j)\right) \tag{4}$$

For a batch of N predictions, the loss is the mean: $\mathcal{L}_{batch} = -\frac{1}{N}\sum_{i=1}^{N} \log(p_{y^{(i)}})$.

Implementation Detail: PyTorch's `F.cross_entropy` expects the logits tensor to be 2-dimensional (N, C) and the targets tensor to be 1-dimensional (N). Because our tensors are of shape (B, T, C) and (B, T), we must reshape (flatten) them to $(B \cdot T, C)$ and $(B \cdot T)$ respectively before calculating the loss.

4 Autoregressive Generation

During training, predictions for all T time steps in a block are computed in parallel. During generation, however, tokens must be sampled sequentially (autoregressively).

To generate text, we:

1. Pass the current context $\mathbf{X}$ into the model to obtain logits $\mathbf{L} \in \mathbb{R}^{B \times T \times C}$.
2. Isolate the logits corresponding to the *last* time step: $\mathbf{l}_{\text{last}} = \mathbf{L}_{:,-1,:} \in \mathbb{R}^{B \times C}$.
3. Apply Softmax to $\mathbf{l}_{\text{last}}$ to obtain a probability distribution $\mathbf{p}$.
4. Sample a token index $x_{\text{new}} \sim \mathbf{p}$ (e.g., using `torch.multinomial`).
5. Append x_{new} to $\mathbf{X}$ and repeat.

5 Worked Examples and Solved Exercises

5.1 Coding Example: Bigram Forward Pass and Loss

Problem: Implement the `BigramLanguageModel` module in PyTorch. Include a `forward` function that takes `idx` (input) and `targets` (optional), computes the logits, and returns the logits alongside the calculated cross-entropy loss.

Solution:

```
        import torch
        import torch.nn as nn
        import torch.nn.functional as F

        class BigramLanguageModel(nn.Module):
        def __init__(self, vocab_size):
        super().__init__()
        # The embedding table serves as our bigram weight matrix W
        self.token_embedding_table = nn.Embedding(vocab_size, vocab_size)

        def forward(self, idx, targets=None):
        # idx and targets are both (B, T) tensors of integers
        # logits will have shape (B, T, C) where C = vocab_size
        logits = self.token_embedding_table(idx)

        if targets is None:
        loss = None
        else:
        # Reshape for PyTorch's cross_entropy function
        B, T, C = logits.shape
        logits = logits.view(B*T, C)    # Shape: (B*T, C)
        targets = targets.view(B*T)     # Shape: (B*T)

        loss = F.cross_entropy(logits, targets)

        return logits, loss

```

5.2 Mathematical Exercise: Cross-Entropy by Hand

Problem: Let our vocabulary size be $V = 3$. For a specific token at step $t-1$, the bigram model outputs the logits $\mathbf{l} = (0.2, 1.5, -0.5)$. The true next token (the target) is $y = 2$ (the second element, 1-indexed). Calculate the Softmax probabilities and the resulting Cross-Entropy Loss.

Solution: Step 1: Calculate the exponentials of the logits.

$$e^{l_1} = e^{0.2} \approx 1.221$$
$$e^{l_2} = e^{1.5} \approx 4.482$$
$$e^{l_3} = e^{-0.5} \approx 0.607$$

Step 2: Calculate the sum of exponentials.

$$\sum e^{l_j} = 1.221 + 4.482 + 0.607 = 6.310$$

Step 3: Calculate the Softmax probabilities.

$$p_1 = 1.221/6.310 \approx 0.193$$
$$p_2 = 4.482/6.310 \approx 0.710$$
$$p_3 = 0.607/6.310 \approx 0.096$$

Notice that $\sum p_j = 0.193 + 0.710 + 0.096 = 0.999 \approx 1.0$.

Step 4: Calculate the Cross-Entropy Loss. The target class is $y = 2$.

$$\mathcal{L} = -\log(p_2) = -\log(0.710) \approx 0.342$$

5.3 Coding Exercise: Implementing the `generate` function

Problem: Extend the `BigramLanguageModel` class with a `generate` function. It should take a sequence of indices `idx` of shape (B, T) and iteratively append `max_new_tokens` to it, returning a tensor of shape $(B, T + \text{max\_new\_tokens})$.

Solution:

```
        # (Inside the BigramLanguageModel class)
        def generate(self, idx, max_new_tokens):
        # idx is (B, T) array of indices in the current context
        for _ in range(max_new_tokens):
        # 1. Get the predictions
        # We don't pass targets, so loss is None
        logits, loss = self(idx)

        # 2. Focus only on the last time step
        # logits shape becomes (B, C)
        logits = logits[:, -1, :]

        # 3. Apply softmax to get probabilities
        probs = F.softmax(logits, dim=-1) # (B, C)

        # 4. Sample from the distribution
        # returns a tensor of shape (B, 1)
        idx_next = torch.multinomial(probs, num_samples=1)

        # 5. Append sampled index to the running sequence
        # Output shape becomes (B, T+1), then (B, T+2)...
        idx = torch.cat((idx, idx_next), dim=1)

        return idx

```